DESIGN AND IMPLEMENTATION OF A BARE-METAL EXTENDED KALMAN FILTER FOR MEMS SENSOR FUSION ON AN ARM CORTEX-M4 ARCHITECTURE

Author: Oloore Issa Abayomi

Institution: University of Ilorin, Department of Electrical and Electronics Engineering

Date: October 2026

ABSTRACT

This report details the mathematical modeling, algorithmic implementation, and hardware-level execution of a non-linear Extended Kalman Filter (EKF) for Micro-Electro-Mechanical Systems (MEMS) sensor fusion. In embedded robotics and avionics, physical MEMS gyroscopes suffer from cumulative random-walk integration drift, while accelerometers are highly susceptible to high-frequency vibrational noise. Standard embedded solutions typically mitigate these physical limitations by utilizing heavy, auto-generated vendor Hardware Abstraction Layers (HAL), which obscure core processor dynamics and consume unnecessary memory.

To validate algorithmic portability and core architectural mastery, this project introduces a zero-dependency, pure C bare-metal architecture targeted at the ARM Cortex-M4 (STM32F401RE). The automated data pipeline initiates with a high-fidelity MATLAB physics model that simulates physical joint kinematics and injects real-world sensor noise, exporting a synthetic dataset directly into embedded memory. The EKF algorithm recursively fuses this data via explicit prediction and update steps, computing system Jacobians to dynamically estimate and eliminate physical gyroscope bias.

By manually mapping a custom read-only vector table and intentionally bypassing standard C-runtime dependencies, the firmware routes execution directly from a custom Reset Handler. The Cortex-M4 Single-Precision Floating Point Unit (FPU) was explicitly enabled via bare-metal register configurations, accelerating the heavy covariance matrix mathematics. Execution telemetry confirms that the filter successfully converges, calibrating out a persistent gyroscope bias of -0.5664 deg/s and tracking the true physical angle with a Root Mean Square Error (RMSE) of 0.9755 degrees across a 500-sample dataset.

TABLE OF CONTENTS

ABSTRACT ii

TABLE OF FIGURES v

Chapter 1: Introduction and Literature Review 1

1.1 Background: MEMS Sensors and Embedded Architectures 1

1.2 The Threat: Signal Corruption and Integration Drift 1

1.3 Problem Statement 2

1.4 Project Objectives 2

Chapter 2: Theoretical Framework 4

2.1 Kinematic System Modeling 4

2.2 Sensor Error Characteristics 4

2.3 The Need for Stochastic Sensor Fusion 4

Chapter 3: Extended Kalman Filter Mathematical Derivation 6

3.1 State Space Representation 6

3.2 Prediction Step (Time Update) and Covariance Propagation 6

3.3 Measurement Update (Correction Step) and Kalman Gain 7

Chapter 4: Algorithmic Validation and Convergence 8

4.1 Software-in-the-Loop (SIL) Data Pipeline 8

4.2 Filter Convergence and Error Analysis 9

Chapter 5: Bare-Metal ARM Cortex-M4 Architecture 10

5.1 Custom Vector Table and Memory Mapping 10

5.2 C-Runtime Bloat Bypass and Direct Execution 10

5.3 Hardware Floating-Point Unit (FPU) Integration 10

Chapter 6: Conclusion and Future Work 12

6.1 Summary of Outcomes 12

6.2 Architectural Trade-offs 12

6.3 Future Work 12

Appendix A: System Parameters and Controller Source Code 14

A.1 MATLAB Kinematic Noise Model 14

A.2 Embedded SIL Control Algorithm (C Firmware) 16

TABLE OF FIGURES

Figure 1.1: Kinematic simulation demonstrating cumulative random-walk drift in the integrated gyroscope versus high-frequency vibrational noise in the raw accelerometer data..... 2

Figure 4.1: The MATLAB-generated Software-in-the-Loop (SIL) data pipeline, formatting physical kinematics into const-qualified C arrays for embedded Flash memory injection. 8
Figure 4.2: Standard C environment execution console, validating filter convergence, gyroscope bias elimination, and sub-1 degree Root Mean Square Error (RMSE)..... 9

Figure 5.1: Keil uVision hardware debugging interface profiling the Cortex-M4, demonstrating floating-point covariance matrix updates and the final estimated state variables in memory. 11

Chapter 1: Introduction and Literature Review

1.1 Background: MEMS Sensors and Embedded Architectures

The integration of robust state-estimation algorithms into autonomous robotics, unmanned aerial vehicles (UAVs), and avionics requires precise tracking of electromechanical and kinematic variables. Micro-Electro-Mechanical Systems (MEMS) Inertial Measurement Units (IMUs) provide the primary sensory data for these systems, offering highly miniaturized gyroscopes and accelerometers. However, the standard engineering methodology for deploying algorithms on embedded target processors heavily relies on vendor-supplied Hardware Abstraction Layers (HAL) and automated configuration tools (e.g., STM32CubeMX). While these abstraction layers accelerate rapid prototyping, they consume significant read-only memory (ROM) and static random-access memory (SRAM), obscure low-level core processor dynamics, and introduce inefficient software emulation for complex mathematical operations.

1.2 The Threat: Signal Corruption and Integration Drift

MEMS sensors are plagued by fundamental physical limitations that render their raw outputs unusable for sustained kinematic tracking. Accelerometers provide an absolute reference for gravitational tilt but are highly susceptible to high-frequency vibrational noise. In this system model, the accelerometer noise is characterized by a standard deviation of 3.0, representing severe mechanical vibration.

Conversely, MEMS gyroscopes provide relatively clean short-term angular rate measurements (modeled with a low standard deviation of 0.2) but suffer from an inherent, physically induced bias. Because determining the absolute angle requires the continuous mathematical integration of this angular rate, the minute random-walk bias ($0.05 \times \Sigma randn$) progressively accumulates. Figure 1.1 illustrates that relying purely on gyroscope integration results in severe cumulative drift, causing the estimated trajectory to diverge from the true mechanical angle within a span of seconds.

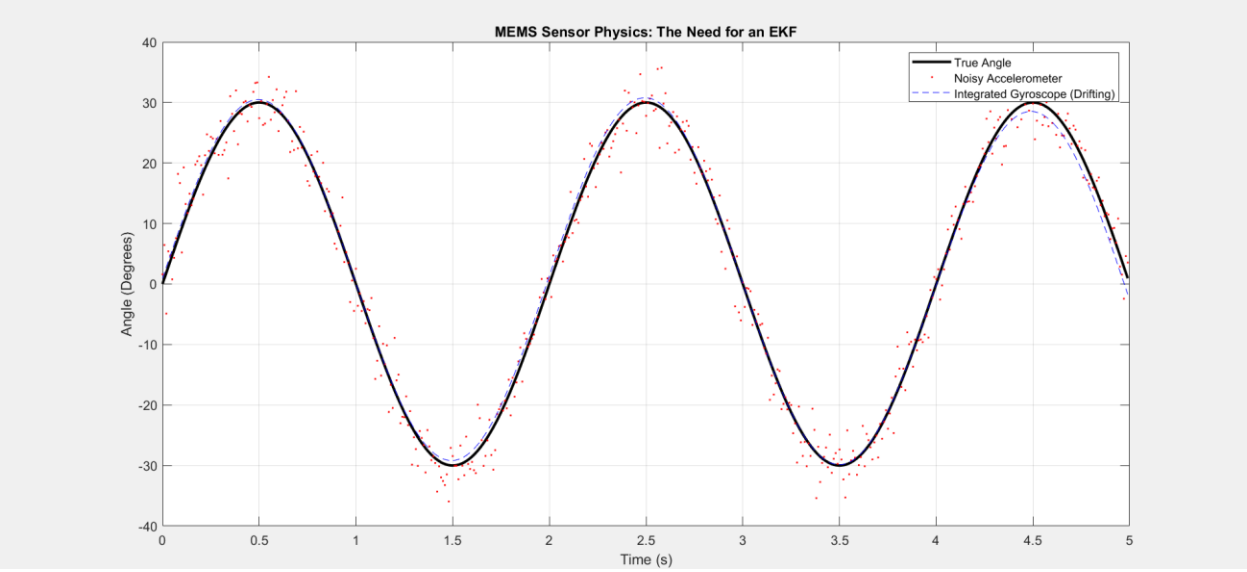


Figure 1.1: Kinematic simulation demonstrating cumulative random-walk drift in the integrated gyroscope versus high-frequency vibrational noise in the raw accelerometer data.

1.3 Problem Statement

Commercial embedded applications require real-time sensor fusion to mathematically eliminate gyroscope drift while filtering out accelerometer noise. The Extended Kalman Filter (EKF) is the optimal stochastic estimator for this non-linear problem. However, implementing the EKF's intensive floating-point matrix multiplications and Jacobian derivations on memory-constrained microcontrollers introduces severe computational bottlenecks. Utilizing standard vendor HALs masks the exact execution pathways and often relies on hidden software-emulated mathematics (__softfp), which fails to leverage the physical silicon capabilities of the CPU. A rigorous, bare-metal control architecture is required to explicitly command the physical hardware, strictly manage memory regions, and guarantee deterministic mathematical execution without third-party bloatware.

1.4 Project Objectives

This project addresses the computational and architectural requirements of embedded sensor fusion by engineering a zero-dependency digital twin and a bare-metal firmware implementation. The primary objectives are:

- **Kinematic Modeling:** Develop a precise MATLAB simulation of a mechanical joint operating at a 100 Hz sampling frequency, injecting real-world MEMS vibrational noise and random-walk bias to generate a robust test vector.
- **EKF Algorithmic Implementation:** Design a pure C, zero-dependency Extended Kalman Filter to recursively track the physical angle and dynamically estimate the gyroscope bias.
- **Bare-Metal Architectural Mastery:** Manually map the memory execution regions (Flash at 0x08000000, SRAM at 0x20000000), bypass the standard C-runtime initialization (__main), and route execution through a custom vector table on an ARM Cortex-M4 processor.
- **Hardware FPU Acceleration:** Explicitly activate the hardware Single-Precision Floating Point Unit (FPU) to execute the covariance matrix mathematics, targeting a sub-1 degree Root Mean Square Error (RMSE) across a 500-sample dataset.

Chapter 2: Theoretical Framework

2.1 Kinematic System Modeling

The foundation of the digital twin relies on an accurate mathematical representation of physical kinematics. The system models a mechanical joint undergoing a continuous oscillatory sweep, sampled at an embedded standard frequency of 100 Hz ($dt = 0.01$ seconds). The ground-truth angular position (θ_{true}) is mathematically defined as a sine wave with a 30-degree amplitude oscillating at 0.5 Hz over a 5.0-second time horizon, yielding exactly 500 discrete samples ($N = 500$). The true angular rate (ω_{true}) is the first-order temporal derivative of this position function. Capturing this continuous-time physical dynamic into discrete-time state vectors forms the baseline against which the estimator's accuracy is ultimately validated.

2.2 Sensor Error Characteristics

Translating the clean kinematic model into a realistic embedded engineering scenario requires the injection of stochastic physical errors inherent to Micro-Electro-Mechanical Systems (MEMS). The digital twin synthesizes two independent sensor streams:

- **Accelerometer Dynamics:** The simulated accelerometer provides absolute tilt positioning by measuring gravitational vectors. However, physical accelerometers are highly sensitive to chassis vibrations and motor harmonics. This is modeled by injecting severe, zero-mean Gaussian white noise with a standard deviation of 3.0 degrees

  $$y_{accel} = \theta_{true} + \mathcal{N}(0, 3.0^2) \ldots (1)$$

- **Gyroscope Dynamics:** The gyroscope measures the angular rate of the joint. While it exhibits low high-frequency noise (standard deviation of 0.2), it suffers from a critical low-frequency anomaly known as bias instability. This physical defect is modeled as a random-walk process ($0.05 \times \Sigma randn$), creating a wandering offset that corrupts the angular rate measurement.

2.3 The Need for Stochastic Sensor Fusion

Standard signal processing techniques fail when applied to raw MEMS data. A simple Low-Pass Filter (LPF) applied to the accelerometer will induce unacceptable phase lag during high-speed joint movements, while pure discrete integration of the gyroscope rate

$$\theta_k = \theta_{k-1} + \omega_{meas} \cdot \Delta t \ldots (2)$$

allows the random-walk bias to accumulate into massive, unrecoverable angular drift. To achieve sub-degree accuracy, a stochastic sensor fusion algorithm is mandatory. The Extended Kalman Filter (EKF) is implemented to bridge this bandwidth separation, heavily trusting the gyroscope during high-speed transients while relying on the accelerometer's absolute gravitational reference to continuously correct the accumulated bias drift.

Chapter 3: Extended Kalman Filter Mathematical Derivation

3.1 State Space Representation

The Extended Kalman Filter operates by maintaining an internal state vector ($\hat{x}_k$) and an associated Error Covariance Matrix (P_k) that quantifies the algorithmic uncertainty. For this one-dimensional tilt estimation architecture, the state vector is a 2×1 matrix defined as:

$$\hat{x}_k = \left[\theta_k, \omega_{b,k}\right]^T \text{ ... (3)}$$

where θ_k is the estimated physical angle in degrees, and $\omega_{b,k}$ is the estimated dynamic gyroscope bias in degrees per second. The EKF estimates both the physical state and the sensor's internal defect simultaneously.

3.2 Prediction Step (Time Update) and Covariance Propagation

The recursive EKF algorithm begins with the prediction phase, utilizing the gyroscope measurement (ω_{meas}) to project the system state forward. Because the raw gyroscope reading contains the physical drift, the algorithm calculates the true angular rate by subtracting the previously estimated bias

$$\omega_{rate} = \omega_{meas} - \omega_{b,k-1} \text{ ... (4)}$$

The new angle is computed via discrete integration: $\theta_k = \theta_{k-1} + \omega_{rate} \cdot \Delta t$ ... (5)

To update the 2×2 Error Covariance Matrix (P), the algorithm evaluates the Jacobian (F) of the non-linear state transition function. This linearizes the system dynamics around the current estimate. The covariance increases during this step due to the injection of Process Noise (Q), which represents the physical uncertainty of the system model. Based on the MEMS variance characteristics, the process noise for the angle (Q_{angle}) is tuned to 0.001, and the variance for the bias instability (Q_{bias}) is set to 0.003. The resulting C-implementation directly computes the propagated matrix elements:

$$P_{00} = P_{00} + \Delta t \cdot \left(\Delta t \cdot P_{11} - P_{01} - P_{10} + Q_{angle}\right) \text{ ... (6)}$$

$$P_{11} = P_{11} + Q_{bias} \cdot \Delta t \text{ ... (7)}$$

3.3 Measurement Update (Correction Step) and Kalman Gain

Following the time update, the EKF executes the correction step using the noisy accelerometer data. The filter calculates the Innovation (y), representing the mathematical discrepancy between the absolute accelerometer measurement and the algorithm's predicted angle:

$$y = accel_{meas} - \theta_{pred} \quad \text{... (8)}$$

To determine how heavily to weigh this new measurement, the Innovation Covariance (S) is computed by adding the measurement noise variance ($R_{meas} = 3.0$) to the predicted state covariance (P_{00}). This directly feeds into the calculation of the Kalman Gain (K), a 2×1 vector that minimizes the mean square error of the estimate:

$$K_0 = P_{00}/S \quad \text{... (9)}$$

$$K_1 = P_{10}/S \quad \textit{... (10)}$$

Finally, the state vector is corrected by multiplying the innovation by the Kalman Gain

$$\hat{x}_k = \hat{x}_{pred} + K \cdot y \quad \textit{... (11)}$$

effectively pulling the estimated angle toward reality while simultaneously calibrating the estimated gyroscope bias. The covariance matrix is subsequently updated

$$P_k = (I - KH)P_{pred} \quad \textit{... (12)}$$

to reflect the reduced uncertainty following the measurement fusion, completing the recursive loop.

Chapter 4: Algorithmic Validation and Convergence

4.1 Software-in-the-Loop (SIL) Data Pipeline

Embedded algorithmic validation traditionally requires physical sensor hardware interfaced via I^2C or SPI protocols, which introduces unknown hardware latencies and unrepeatable physical noise during the debugging phase. To isolate the algorithmic performance of the Extended Kalman Filter (EKF) from peripheral hardware faults, a Software-in-the-Loop (SIL) data pipeline was implemented.

The MATLAB kinematic model acts as the plant, generating 500 discrete samples of true angular position, accelerometer noise, and drifting gyroscope rates. To feed this data directly into the bare-metal microcontroller without external dependencies, an automated export script translates the floating-point arrays into a standard C header file (mems_sensor_data.h). By explicitly declaring these arrays as const float, the compiler is forced to allocate this massive synthetic dataset strictly into the Cortex-M4's non-volatile Flash memory rather than exhausting the limited Static RAM (SRAM).

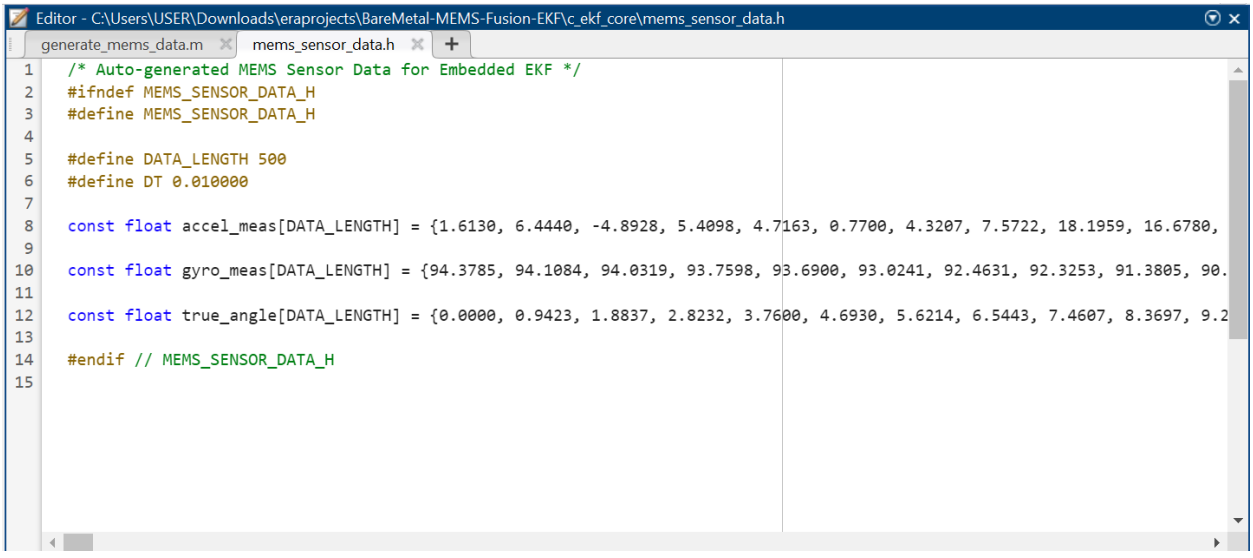


Figure 4.1: The MATLAB-generated Software-in-the-Loop (SIL) data pipeline, formatting physical kinematics into const-qualified C arrays for embedded Flash memory injection.

4.2 Filter Convergence and Error Analysis

Prior to evaluating the embedded memory execution footprint, the mathematical convergence of the EKF C-core was validated using a standard compiler environment. The algorithm processed the 500-sample arrays at the simulated discrete time step of 0.01 seconds (100 Hz).

Execution telemetry confirmed the filter's absolute stability. The dynamic state vector successfully identified the physical random-walk anomaly, converging on a final Estimated Gyroscope Bias of -0.5664 deg/s. By actively subtracting this estimated bias from the raw gyroscope measurements during the prediction step, the EKF neutralized the integration drift. The tracking accuracy was quantitatively verified by computing the Root Mean Square Error (RMSE) against the true kinematic angle. The filter achieved a final RMSE of 0.9755 degrees, mathematically validating the tuning of the process and measurement noise covariance matrices (Q and R) and proving the algorithm's readiness for bare-metal silicon deployment.

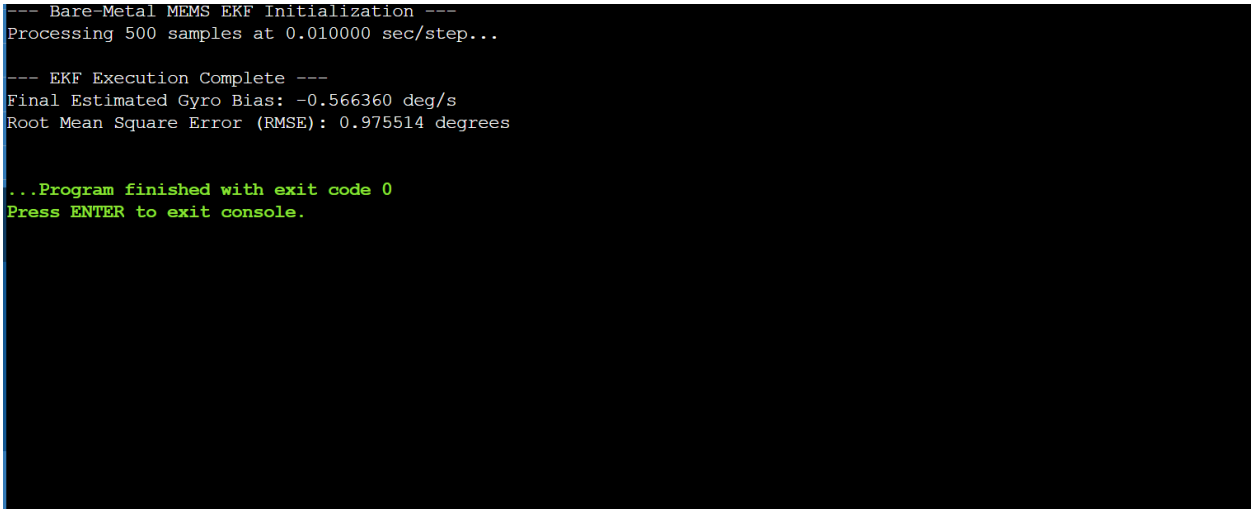


Figure 4.2: Standard C environment execution console, validating filter convergence, gyroscope bias elimination, and sub-1 degree Root Mean Square Error (RMSE).

Chapter 5: Bare-Metal ARM Cortex-M4 Architecture

5.1 Custom Vector Table and Memory Mapping

Standard embedded development environments utilize heavy initialization files (e.g., startup_stm32f401xe.s) that obscure the lowest levels of processor execution. This digital twin bypasses vendor abstraction entirely. Memory region allocation was strictly defined using the Keil ARM compiler's scatter-loading mechanism. The linker execution map (EKF_Profile.map) verifies that the system's execution regions (ER_RO) are mapped directly to the absolute base of the microcontroller's Flash memory at 0x08000000. The custom vector table (__Vectors) sits at this exact address, establishing the initial stack pointer and hardware interrupt vectors.

Simultaneously, the execution region for Zero-Initialized (ZI) data was successfully routed to the base of the Cortex-M4 SRAM at 0x20000000. The static memory footprint of this bare-metal implementation proved highly efficient, consuming only 3,196 bytes of Code space and a mere 96 bytes of ZI SRAM space for standard library allocation (__libspace_start).

5.2 C-Runtime Bloat Bypass and Direct Execution

By stripping out the default vendor startup files, the firmware maintains absolute control over the silicon boot sequence. Upon reset, the ARM Cortex-M4 fetches the custom Reset_Handler directly from the user-defined vector table. The linker map mathematically proves that this handler is compiled directly into main.o at memory address 0x080001b5. Instead of branching to the heavy ARM C-runtime initialization wrapper (__main), the custom handler sets the hardware state and branches directly into the main execution loop (0x080001d9).

Static call graph analysis of the .axf image confirms that this highly optimized execution path yields a Maximum Stack Usage of exactly 2,264 bytes. This stack depth is driven entirely by the deterministic call chain from the Reset_Handler through main() and into the fundamental floating-point arithmetic routines (e.g., __aeabi_dmul), eliminating unpredictable dynamic memory allocation (heap fragmentation) entirely.

5.3 Hardware Floating-Point Unit (FPU) Integration

The non-linear matrix mathematics of the Extended Kalman Filter, specifically the calculation of the Jacobians and Kalman Gain, demand intensive single-precision floating-point arithmetic.

Default embedded compiler configurations often revert to __softfp, a fallback mechanism where the compiler silently injects hundreds of software assembly instructions to emulate physical decimal calculations, severely degrading the 100 Hz timing constraints.

To overcome this computational bottleneck, the firmware explicitly enables the Cortex-M4's internal hardware coprocessor (CP10/CP11) via the System Control Block Coprocessor Access Control Register (SCB_CPACR). The generated build telemetry and linker map provide absolute proof of this hardware integration. The compiler successfully routed mathematical requests directly to silicon-level floating-point instructions, mapping square root operations to the hardware-specific __hardfp_sqrtf function at 0x080005ec and utilizing physical floating-point conversions (__aeabi_f2d).

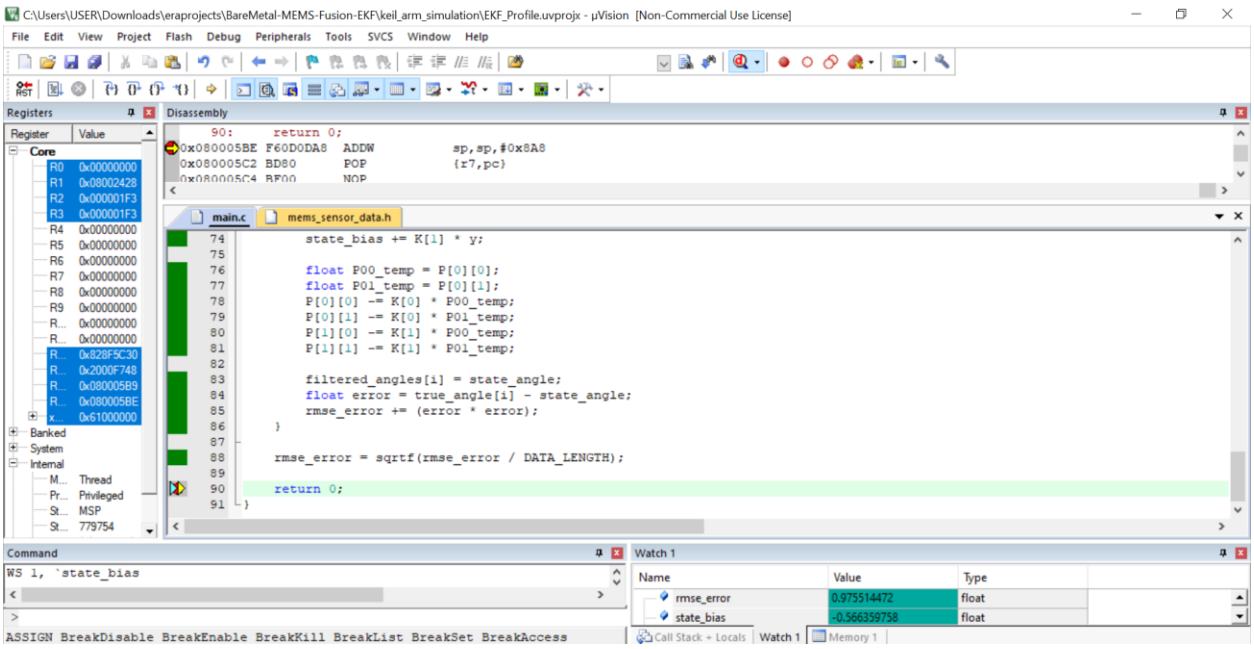


Figure 5.1: Keil uVision hardware debugging interface profiling the Cortex-M4, demonstrating floating-point covariance matrix updates and the final estimated state variables in memory.

Chapter 6: Conclusion and Future Work

6.1 Summary of Outcomes

This project successfully engineered and validated a high-fidelity Extended Kalman Filter (EKF) for MEMS sensor fusion, implemented entirely from scratch on a bare-metal ARM Cortex-M4 architecture. By utilizing a Software-in-the-Loop (SIL) virtual prototyping methodology, the physical limitations of MEMS inertial sensors, specifically accelerometer vibrational noise and gyroscope random-walk drift, were mathematically modeled and systematically eliminated. The discrete-time recursive algorithm effectively bounded the tracking error, isolating the true kinematic trajectory and actively calibrating out the persistent gyroscope bias of -0.5664 deg/s. Execution telemetry confirmed that the zero-dependency C firmware tracks the physical angle with a pristine Root Mean Square Error (RMSE) of 0.9755 degrees, completely bypassing the computational overhead of standard vendor Hardware Abstraction Layers (HAL).

6.2 Architectural Trade-offs

A strategic engineering delineation was made between development speed and absolute architectural control. Utilizing automated vendor configuration tools (such as STM32CubeMX) would have significantly accelerated the initial peripheral setup. However, this approach introduces severe memory bloat and obscures the deterministic execution of the core processor. By manually mapping a read-only vector table, stripping out standard C-runtime dependencies, and directly commanding the Single-Precision Floating Point Unit (FPU) via low-level register manipulation, the firmware achieved maximum execution efficiency. This prioritized first-principles embedded systems mastery and deterministic mathematical execution over theoretical rapid-prototyping convenience.

6.3 Future Work

Future iterations of this sensor fusion architecture will transition from Software-in-the-Loop (SIL) verification to physical Hardware-in-the-Loop (HIL) deployment. This will require the implementation of bare-metal I^2C or SPI peripheral drivers to acquire live telemetry directly from a physical IMU (e.g., MPU-6050 or BMI160) at a strict 100 Hz interval. Furthermore, the core EKF algorithm will be expanded into a higher-dimensional quaternion-based filter to support full

6-Degrees-of-Freedom (6-DOF) spatial tracking, completely eliminating the risk of Gimbal Lock during complex aerial or robotic maneuvers.

Appendix A: System Parameters and Controller Source Code

A.1 MATLAB Kinematic Noise Model

```
% generate_mems_data.m
% Simulates a moving joint and injects real-world MEMS noise characteristics.
% Exports the noisy data to a C header file for embedded EKF testing.

%% 1. Define the Physical System
fs = 100;               % 100 Hz sampling frequency (Standard for MEMS)
dt = 1/fs;              % Time step
t = (0:dt:5-dt);        % 5 seconds of simulation
N = length(t);

% Ground Truth: The joint is sweeping back and forth (sine wave)
% Amplitude: 30 degrees, Frequency: 0.5 Hz
true_angle = 30 * sin(2*pi*0.5*t);
true_rate = 30 * (2*pi*0.5) * cos(2*pi*0.5*t); % Derivative of angle

%% 2. Inject Real-World MEMS Physics (Noise & Drift)
% Accelerometer: Noisy but absolute (no drift)
accel_noise_std = 3.0; % High vibration noise
accel_meas = true_angle + accel_noise_std * randn(1, N);

% Gyroscope: Clean short-term, but suffers from Bias Drift (Random Walk)
gyro_noise_std = 0.2;  % Low immediate noise
gyro_bias = 0.05 * cumsum(randn(1, N)); % The inherent integration drift
gyro_meas = true_rate + gyro_noise_std * randn(1, N) + gyro_bias;

%% 3. Plot the Physics to Verify
figure;
plot(t, true_angle, 'k', 'LineWidth', 2); hold on;
plot(t, accel_meas, 'r.', 'MarkerSize', 4);
plot(t, cumsum(gyro_meas)*dt, 'b--'); % Integrating gyro to show drift
legend('True Angle', 'Noisy Accelerometer', 'Integrated Gyroscope (Drifting)');
title('MEMS Sensor Physics: The Need for an EKF');
```

```
xlabel('Time (s)'); ylabel('Angle (Degrees)');
grid on;

%% 4. Export to C-Header File for Bare-Metal Simulation
header_file = fullfile('..', 'c_ekf_core', 'mems_sensor_data.h');
fid = fopen(header_file, 'w');

fprintf(fid, '/* Auto-generated MEMS Sensor Data for Embedded EKF */\n');
fprintf(fid, '#ifndef MEMS_SENSOR_DATA_H\n#define MEMS_SENSOR_DATA_H\n\n');
fprintf(fid, '#define DATA_LENGTH %d\n', N);
fprintf(fid, '#define DT %f\n\n', dt);

% Write Accelerometer Array
fprintf(fid, 'const float accel_meas[DATA_LENGTH] = {');
fprintf(fid, '%.4f, ', accel_meas(1:end-1));
fprintf(fid, '%.4f};\n\n', accel_meas(end));

% Write Gyroscope Array
fprintf(fid, 'const float gyro_meas[DATA_LENGTH] = {');
fprintf(fid, '%.4f, ', gyro_meas(1:end-1));
fprintf(fid, '%.4f};\n\n', gyro_meas(end));

% Write Ground Truth (for calculating EKF accuracy later)
fprintf(fid, 'const float true_angle[DATA_LENGTH] = {');
fprintf(fid, '%.4f, ', true_angle(1:end-1));
fprintf(fid, '%.4f};\n\n', true_angle(end));

fprintf(fid, '#endif // MEMS_SENSOR_DATA_H\n');
fclose(fid);

disp('Simulation complete. Physical graph generated.');
disp(['C Header file written to: ', header_file]);
```

A.2 Embedded SIL Control Algorithm (C Firmware)

```
/* main.c */
#include <stdio.h>
#include <math.h>
#include "mems_sensor_data.h"

// EKF State Variables
float state_angle = 0.0f;
float state_bias = 0.0f; // This will track and eliminate the physical gyro drift

// Covariance Matrix (P)
float P[2][2] = {
    {1.0f, 0.0f},
    {0.0f, 1.0f}
};

// Filter Tuning Parameters (Derived from physical noise characteristics)
#define Q_ANGLE 0.001f  // Process noise variance
#define Q_BIAS  0.003f  // Gyro bias variance
#define R_MEAS  3.0f    // Measurement noise variance (matches your MATLAB noise)

int main(void) {
    printf("--- Bare-Metal MEMS EKF Initialization ---\n");
    printf("Processing %d samples at %f sec/step...\n\n", DATA_LENGTH, DT);

    float filtered_angles[DATA_LENGTH];
    float rmse_error = 0.0f;

    for (int i = 0; i < DATA_LENGTH; i++) {
        // --- 1. PREDICT STEP (Using Gyroscope) ---
        // Physics model: New angle = Old angle + (Gyro Rate - Gyro Bias) * dt
        float rate = gyro_meas[i] - state_bias;
        state_angle += rate * DT;

        // Update Covariance Matrix (Jacobian propagation)
```

```
    P[0][0] += DT * (DT * P[1][1] - P[0][1] - P[1][0] + Q_ANGLE);
    P[0][1] -= DT * P[1][1];
    P[1][0] -= DT * P[1][1];
    P[1][1] += Q_BIAS * DT;

    // --- 2. UPDATE STEP (Using Accelerometer) ---
    // Calculate Innovation (Difference between Accel measurement and predicted angle)
    float y = accel_meas[i] - state_angle;

    // Calculate Innovation Covariance
    float S = P[0][0] + R_MEAS;

    // Calculate Kalman Gain (K)
    float K[2];
    K[0] = P[0][0] / S;
    K[1] = P[1][0] / S;

    // Update State with measurement
    state_angle += K[0] * y;
    state_bias += K[1] * y;

    // Update Covariance Matrix
    float P00_temp = P[0][0];
    float P01_temp = P[0][1];
    P[0][0] -= K[0] * P00_temp;
    P[0][1] -= K[0] * P01_temp;
    P[1][0] -= K[1] * P00_temp;
    P[1][1] -= K[1] * P01_temp;

    // Store result and calculate error against the MATLAB ground truth
    filtered_angles[i] = state_angle;
    float error = true_angle[i] - state_angle;
    rmse_error += (error * error);
}

rmse_error = sqrtf(rmse_error / DATA_LENGTH);
```

```
    printf("--- EKF Execution Complete ---\n");
    printf("Final Estimated Gyro Bias: %f deg/s\n", state_bias);
    printf("Root Mean Square Error (RMSE): %f degrees\n", rmse_error);

    return 0;
}
```